

Створюємо таблицю:

```
(smth@ smth)~[~]
$ aws dynamodb create-table --table-name WinaData --attribute-definitions AttributeName=Class,AttributeType=N --key
-schema AttributeName=Class,KeyType=HASH --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5
{
  "TableDescription": {
    "AttributeDefinitions": [
      {
        "AttributeName": "Class",
        "AttributeType": "N"
      }
    ],
    "TableName": "WinaData",
    "KeySchema": [
      {
        "AttributeName": "Class",
        "KeyType": "HASH"
      }
    ],
    "TableStatus": "CREATING",
    "CreationDateTime": "2024-06-29T00:25:43.842000-04:00",
    "ProvisionedThroughput": {
      "NumberOfDecreasesToday": 0,
      "ReadCapacityUnits": 5,
      "WriteCapacityUnits": 5
    },
    "TableSizeBytes": 0,
    "ItemCount": 0,
    "TableArn": "arn:aws:dynamodb:eu-north-1:891377288524:table/WinaData",
    "TableId": "fd20b846-8837-4134-a54e-dd8ed0dcc39d",
    "DeletionProtectionEnabled": false
  }
}
```

Бачимо її у AWS:

Tables (1) Info

Find tables by table name

Any tag key

Any tag value

< 1 >

☐	Name	Status	Partition key	Sort key	Indexes	Deletion protection	Read capacity mode	Write capacity mode	Total size	Table class
☐	WinaData	Active	Class (N)	-	0	Off	Provisioned (5)	Provisioned (5)	0 bytes	Standard

Створюємо по семплу в AWS CLI та AWS Management Console:

Attributes View DynamoDB JSON

```
1 {
2   "Class": {
3     "N": "2"
4   },
5   "Alcalinity of ash": {
6     "N": "11.6"
7   },
8   "Alcohol": {
9     "N": "12.23"
10  },
11  "Ash": {
12    "N": "2.4"
13  },
14  "Color intensity": {
15    "N": "5.54"
16  },
17  "Flavanoids": {
18    "N": "3.06"
19  },
20  "Hue": {
21    "N": "1.14"
22  },
23  "Magnesium": {
24    "N": "120"
25  }
26 }
```

JSON Ln 1, Col 1 0 Errors: 0 0 Warnings: 0

```
(smth@smth)-[~]
$ aws dynamodb put-item --table-name WinaData --item '{"Class": {"N": "1"},
  "Alcohol": {"N": "14.23"},
  "Malic acid": {"N": "1.71"},
  "Ash": {"N": "2.43"},
  "Alcalinity of ash": {"N": "15.6"},
  "Magnesium": {"N": "127"},
  "Total phenols": {"N": "2.8"},
  "Flavanoids": {"N": "3.06"},
  "Nonflavanoid phenols": {"N": "0.28"},
  "Proanthocyanins": {"N": "2.29"},
  "Color intensity": {"N": "5.64"},
  "Hue": {"N": "1.04"},
  "OD280/OD315 of diluted wines": {"N": "3.92"},
  "Proline": {"N": "1065"}}'
```

Пошук даних:

```
(smth@smth)-[~]
$ aws dynamodb get-item --table-name WinaData --key '{"Class": {"N": "1"}}'
{
  "Item": {
    "Class": {
      "N": "1"
    },
    "Flavanoids": {
      "N": "3.06"
    },
    "Alcohol": {
      "N": "14.23"
    },
    "Proanthocyanins": {
      "N": "2.29"
    },
    "Total phenols": {
      "N": "2.8"
    },
    "Color intensity": {
      "N": "5.64"
    },
    "Proline": {
      "N": "1065"
    },
    "Nonflavanoid phenols": {
      "N": "0.28"
    },
    "OD280/OD315 of diluted wines": {
      "N": "3.92"
    },
    "Alcalinity of ash": {
      "N": "15.6"
    },
    "Ash": {
      "N": "2.43"
    },
    "Hue": {
      "N": "1.04"
    }
  }
}
```

▼ Scan or query items

☒ Scan

☐ Query

Select a table or index

Table - WinaData ▼

Select attribute projection

Specific attributes ▼

Specific attributes to project

Q Class X

Add attribute

▼ Filters

Attribute name

Q Class X

Type

Number ▼

Condition

Equal to ▼

Value

2

Remove

Add filter

Run

Reset

Їх модифікація:

Attribute name	Value	Type	
Flavanoids	3.06	Number	Remove
Hue	1.14	Number	Remove
Magnesium	120	Number	Remove
Malic acid	1.78	Number	Remove
Nonflavanoid phenols	0.38	Number	Remove
OD280/OD315 of diluted wii	4.29	Number	Remove
Proanthocyanins	2.19	Number	Remove
Proline	1165	Number	Remove
Total phenols	4	Number	Remove

Cancel

Save

Save and close

```
(smth@smth)-[~]  
$ aws dynamodb update-item --table-name WinaData --key '{"Class": {"N": "1"}}' --update-expression "set Alcohol = :a" --expression-attribute-values '{"a": {"N": "17"}}'
```

Та видалення:

```
(smth@smth)-[~]  
$ aws dynamodb delete-item --table-name WinaData --key '{"Class": {"N": "2"}}'
```

```

import boto3
from botocore.exceptions import NoCredentialsError, PartialCredentialsError
import json

dynamodb = boto3.resource('dynamodb', region_name='eu-north-1')

def create_table():
    try:
        table = dynamodb.create_table(
            TableName='WineData',
            KeySchema=[
                {
                    'AttributeName': 'Class',
                    'KeyType': 'HASH'
                },
                {
                    'AttributeName': 'ID',
                    'KeyType': 'RANGE'
                }
            ],
            AttributeDefinitions=[
                {
                    'AttributeName': 'Class',
                    'AttributeType': 'N'
                },
                {
                    'AttributeName': 'ID',
                    'AttributeType': 'N'
                }
            ],
            ProvisionedThroughput={
                'ReadCapacityUnits': 5,
                'WriteCapacityUnits': 5
            }
        )
        table.wait_until_exists()
        print("Table created successfully.")
    except Exception as e:
        print(f"Error creating table: {e}")

def insert_data():
    table = dynamodb.Table('WineData')
    try:
        table.put_item(
            Item={
                'Class': 1,
                'ID': 1,
                'Alcohol': 14.23,
                'Malic acid': 1.71
            }
        )
        table.put_item(
            Item={
                'Class': 1,
                'ID': 2,
                'Alcohol': 13.20,
                'Malic acid': 1.78
            }
        )
        print("Data inserted successfully.")
    except Exception as e:
        print(f"Error inserting data: {e}")

def read_data():
    table = dynamodb.Table('WineData')

```

```

try:
    response = table.get_item(
        Key={
            'Class': 1,
            'ID': 1
        }
    )
    item = response.get('Item')
    print(item)
except Exception as e:
    print(f"Error reading data: {e}")

def update_data():
    table = dynamodb.Table('WineData')
    try:
        table.update_item(
            Key={
                'Class': 1,
                'ID': 1
            },
            UpdateExpression='SET Alcohol = :val1',
            ExpressionAttributeValues={
                ':val1': 15.0
            }
        )
        print("Data updated successfully.")
    except Exception as e:
        print(f"Error updating data: {e}")

def delete_data():
    table = dynamodb.Table('WineData')
    try:
        table.delete_item(
            Key={
                'Class': 1,
                'ID': 1
            }
        )
        print("Data deleted successfully.")
    except Exception as e:
        print(f"Error deleting data: {e}")

def load_json_data(file_name):
    table = dynamodb.Table('WineData')
    try:
        with open(file_name) as json_file:
            wines = json.load(json_file)
            for wine in wines:
                table.put_item(Item=wine)
            print("Data loaded from JSON file successfully.")
    except Exception as e:
        print(f"Error loading data from JSON file: {e}")

def query_data():
    table = dynamodb.Table('WineData')
    try:
        response = table.query(
            KeyConditionExpression=boto3.dynamodb.conditions.Key('Class').eq(1)
        )
        items = response.get('Items')
        for item in items:
            print(item)
    except Exception as e:
        print(f"Error querying data: {e}")

```

```

def scan_data():
    table = dynamodb.Table('WineData')
    try:
        response = table.scan()
        items = response.get('Items')
        for item in items:
            print(item)
    except Exception as e:
        print(f"Error scanning table: {e}")

def delete_table():
    try:
        table = dynamodb.Table('WineData')
        table.delete()
        table.wait_until_not_exists()
        print("Table deleted successfully.")
    except Exception as e:
        print(f"Error deleting table: {e}")

create_table()
insert_data()
read_data()
update_data()
delete_data()
load_json_data('/home/smith/Downloads/relaxed.json')
query_data()
scan_data()
delete_table()

```